MPC-PiNNs technique applied to Quadrotors

Authors

January 26, 2025

Todo list

Contents

1 Introduction

Model Predictive Control (MPC) has emerged as a powerful control strategy in robotics thanks to its ability to handle constraints and optimize system behavior over a prediction horizon. When combined with neural networks, which can learn complex system dynamics, it creates a promising framework for controlling nonlinear systems like UAVs.
This report presents the implementation and analysis of a Neural Model Predictive Control (NMPC) scheme applied to a quadrotor. We followed a bottom-up approach, testing the pipeline first on a Unicycle, which still has a nonlinear dynamics, but is easier to treat due a simpler model and faster prototyping possibilities. Our approach integrates Physical Informed Neural Network to approximate the quadrotor's dynamics, which are then utilized within an MPC framework to generate optimal control inputs. We explore the effectiveness of this method in trajectory tracking tasks, comparing it with traditional control approaches.
The following sections detail the theoretical foundations, implementation specifics, and experimental results of our neural MPC system. We also discuss the challenges encountered and potential improvements for future work.

2 Models

Our scheme uses the MPC to generate optimal control inputs while the Neural part is used to model the dynamics and kinematics of our system. The Unicycle has been modeled with its kinematic model while for the quadrotor we have used its dynamic model.

2.1 Unicycle kinematic model

The kinematic model of the unicyle is a simplified representation of the movement of the vehicle in terms of position and orientation. This simple model has been derived directly from the so-called "rolling without slipping constraint". The state space is represented by the robot cartesian position and orientation $[x, y, \theta]$, while the input actions are represented by the driving and steering velocities the vehicle can assume $[v, \omega]$. The kinematic model of a univcyle robot it is described by the following equations:

$$\begin{cases} \dot{x}(t) = v(t)\cos(\theta(t)) \\ \dot{y}(t) = v(t)\sin(\theta(t)) \\ \dot{\theta}(t) = \omega(t) \end{cases} \tag{1}$$

2.2 Quadrotor dynamic model

For the quadrotor, we have used its dynamic model. We can describe this second order model by starting from the two following equations:

$$\begin{cases} m\ddot{p} = f_t + f_g + f_a + f_d \\ J\dot{\omega} + \omega \times (J\omega) = \tau \end{cases} \tag{2}$$

where the first equation, which is derived from Newton's Second Law, represents the Translational Dynamics while the second one, derived from Euler's equations, represents the Rotational Dynamics. In the first equation f_t represents the total thrust force: $f_t = R \cdot \begin{bmatrix} 0 & 0 & T \end{bmatrix}^T$, where R is the rotation matrix of (ϕ, θ, ψ), the roll, pitch and yaw angles. The total thrust force T produced by the rotors is defined by the following equation:

$$T = f_1 + f_2 + f_3 + f_4, \quad \text{with} \quad f_i = k_f \omega_i^2 \tag{3}$$

where k_f is the thrust coefficient and ω_i is the angular speed of the i-th rotor. For the Rotational Dynamics, τ represents the torques in the body frame and can be expressed as follows:

$$\tau = \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} L(f_1 - f_3) \\ L(f_2 - f_4) \\ \tau_\psi \end{bmatrix} \tag{4}$$

where L is the distance from the center of mass of each rotor.
Given the previous equations, if we assume aereodynamics and gyroscopic effects to be negligible and that we have small roll and pitch angles, we can derive the simplified model fo the quadrotor:

$$\begin{cases} \ddot{x} = -(\cos\psi \sin\theta \cos\phi + \sin\psi \sin\phi)\frac{T}{m} \\ \ddot{y} = -(\sin\psi \sin\theta \cos\phi - \cos\psi \sin\phi)\frac{T}{m} \\ \ddot{z} = -(\cos\theta \cos\phi)\frac{T}{m} + g \\ \ddot{\phi} = \frac{\tau_\phi}{I_x} \\ \ddot{\theta} = \frac{\tau_\theta}{I_y} \\ \ddot{\psi} = \frac{\tau_\psi}{I_z} \end{cases} \tag{5}$$

3 Model Predictive Control

Model Predictive Control (MPC), it is an optimal control technique where the actions are chosen solving an optimization problem, composed by a set of constraints

and an objective function to be minimized over a finite horizon.

$$\min_{u(0),u(1),\ldots,u(N-1)} \quad \sum_{k=0}^{N-1} \ell(x(k),u(k)) + \ell_f(x(N)) \tag{6}$$

$$\text{s.t.} \quad x(k+1) = f(x(k),u(k)), \quad k = 0,\ldots,N-1, \tag{7}$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0,\ldots,N-1, \tag{8}$$

$$x(0) = x_0 \tag{9}$$

Here, $\mathcal{U}$ and $\mathcal{X}$ denote the admissible sets for inputs and states, respectively, while x_0 is the initial state. One of the main advantages of this control approach is the ability to handle multivariable interactions and constraints. However, solving the above optimization problem in real time can be computationally challenging, especially for large-scale or highly nonlinear systems. This is one reason why neural networks have been employed to either approximate the system model or enhance the optimization process. In our case, we have used Neural Networks to predict the next state after the application of the control input given by the MPC.

3.1 Cost function

Our objective is to minimize at each step the error with respect to the reference trajectory, both in traslational and orientational dimension, while also maintaining the module of the input actions as low as possible. A terminal state term is present too, where the traslational and orientational errors of the last horizon step are tried to be minimized. Thus, the cost function is composed by the following three terms:

1. **Spatial error**: at each step of the horizon, the squared errors over the state variables are summed together

$$J_{sp_k} = W_x \sum_{i=0}^{n} x_{i_k}^2 \tag{10}$$

   where x_{i_k} it is the i-th component of the state vector at the k-th instant.

2. **Terminal state**: to make sure the final state over the horizon is good, we add a related penalty term in the terminal state N:

$$J_{sp_N} = W_x \sum_{i=0}^{n} x_{i_N}^2 \tag{11}$$

   where x_{i_k} it is the i-th component of the state vector at the last instant over the horizon

3. **Input**: it penalizes high values of input controls

$$J_{u_k} = W_u \sum_{i=0}^{n} u_{i_k}^2 \tag{12}$$

where u_{i_k} it is the i-th component of the control vector at the k-th instant.

3.2 Constraints

The constraints of the NLP concerns some limitations on the state of the vehicle model and on the dynamics of the system. In detail, we have just two constraints:

1. **Initial state**: the initial state of the unicycle model must be the current measured state. Therefore, defining the vehicle state as x:

$$x_0 = x_{current} \tag{13}$$

2. **Dynamics**: given the robot state at a certain time-step as x_t, an input at the same time-step as u_t and the dynamic model of the system defined as $\mathrm{f_{expl}}$, we establish that the next state is obtained by applying the input at the given state according to dynamic model:

$$x_{t+1} = \mathrm{f_{expl}}(x_t, u_t) \tag{14}$$

3.3 Unicycle

For our formulation, we have considered a discrete-time version of the model equations (1) obtained via Euler integration:

$$\begin{cases} x_{k+1} = x_k + T_s\, v_k \cos\theta_k, \\ y_{k+1} = y_k + T_s\, v_k \sin\theta_k, \\ \theta_{k+1} = \theta_k + T_s\, \omega_k. \end{cases} \tag{15}$$

According to this, the cost function terms are the following ones:

1. **Spatial error**: at each step of the horizon, the squared errors along x-axis, y-axis and error of orientation are summed together

$$J_{sp} = W_{e_x} e_x^2 + W_{e_y} e_y^2 + W_{e_\theta} e_\theta^2 \tag{16}$$

where

$$e_x = x - x_{ref} \tag{17a}$$
$$e_y = y - y_{ref} \tag{17b}$$
$$e_\theta = \theta - \theta_{ref} \tag{17c}$$

2. **Terminal state**: to make sure the final stage is a good one, we add a penalty on errors along x-axis, y-axis and error of orientation in the terminal state

$$J_{term} = W_{e_{x_N}} e_{x_N}^2 + W_{e_{y_N}} e_{y_N}^2 + W_{e_{\theta_N}} e_{\theta_N}^2 \tag{18}$$

3. **Input**: it penalizes high values of input controls

$$J_u = W_v v^2 + W_\omega \omega^2 \tag{19}$$

3.4 Quadrotor

Our work is based on a Crazyflie, which is a versatile open source quadrotor that only weighs 27g and fits in the palm of your hand. This particular quadrotor is characterized by a firmware that takes as inputs roll, pitch and yaw desired poses instead of control torques. This happens beacuse there is a low level attitude controller which performs this mapping for us. As a consequence, we have used a first order dynamic model for the MPC formulation.

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{z} = v_z \\ \dot{v}_x = -(\cos\psi \sin\theta \cos\phi + \sin\psi \sin\phi)\frac{T}{m} \\ \dot{v}_y = -(\sin\psi \sin\theta \cos\phi - \cos\psi \sin\phi)\frac{T}{m} \\ \dot{v}_z = -(\cos\theta \cos\phi)\frac{T}{m} + g \\ \dot{\phi} = \frac{\phi_c - \phi}{\tau} \\ \dot{\theta} = \frac{\theta_c - \theta}{\tau} \\ \dot{\psi} = \frac{\psi_c - \psi}{\tau} \end{cases} \tag{20}$$

In this models the state is composed by the cartesian position, the cartesian velocities and the roll pitch yaw angles, while the control inputs are composed by the desidered poses relative to the roll, pitch, yaw angles and the thrust.

$$\mathcal{X} = \begin{bmatrix} x & y & z & v_x & v_y & v_z & \phi & \theta & \psi \end{bmatrix}^T \tag{21}$$

$$\mathcal{U} = \begin{bmatrix} \phi_c & \theta_c & \psi_c & T \end{bmatrix}^T \tag{22}$$

4 Dataset

The data the we used to train our neural models come from simulation and some real world experiments. Generally speaking, we can describe each our dataset as a tuple in the form $\{[x_t, u_t], [x_{t+1}]\}$, were x_t is the state of the system at time t, u_t is the control input at time t and x_{t+1} is the state of the system at time $t+1$.

For the baseline (*simple-neural-network*), no constraints on the dataset form are required, while on the physics informed models specific requirements are needed. In order to apply the PiNN framework with control, [1] has shown that the input needs to be constant in a sub-interval $[0, T]$. This assumption brings us to generate several trajectories with this specific constraint, and also limits the possibility of application on the data obtained in the real-world experiment. More details about this constraint are explored in the Pinn-section.

All the sintetic data has been obtained by integrating the system dynamics with a Runge-kutta 4th order method, while the real-world data has been extrapolated using the ROS interface of the quadrotor.
When extracting data from the real-world experiments (or the gazebo simulations), state and action publishing rates were a crucial aspect to take into account. Especially, when extracting data from the real drone executing a trajectory tracking routine by using an existing MPC controller implented by us, the action publishing rate should be high enough to make the controller safe enough. Similarly, the state publishing rate should be high, but it couldn't be too high, for communication latency reasons, which would have made the use of the real crazyflie impossible. These problems could be partially mitigated by using the gazebo simulator and setting ad-hoc publishing rates, or forcing the input to be constant (introducing a noisy approximation) for a certain amount of time in the real-world data.

foto dataset ?

5 Neural models

The model must be well-defined to be used in NMPC, in such a way to obtain reliable predictions of system behavior in response to various control actions. Neural networks are universal approximators, and they can be used both for designing the entire dynamic model of a system and approximating the disturbances and noises a system can be affected. Our purpose is to approximate the entire dynami model of a quadrotor with a neural model, by first testing the performances on a unicycle model due to its low complexity. In this section we will start describing the used

architecture used

5.0.1 Architecture

The general architecture of the networks is a MLP that takes as input the state of the system, the input and (in the PINN models) the time stamp. We tested the proposed approach with a various number of hidden layers and neurons, keeping the dimension of the network reasonably small for enhancing the possibility to use the on-board computer of the Quadrotor.

In the standard neural network, a smaller number of neurons was sufficient for exploit the dynamics, while the PINN framework required a larger number of neurons for smooth the loss function. Having a smoother loss landscape is crucial for let the model mimic the real dynamics.
In general, the combination of small hidden state and the absence of regularization terms in the standard models are the reason for which the general nerual network framework is not able to approximate the underlying physics laws over the entire training workspace.

5.1 Standard neural network

5.1.1 Training scheme

As can be noticed in the previous section, the dataset does not contain the dynamics of the system, but just the evolution of the states. On the contrary, the forward pass of the network returns the vehicle dynamics. So, what we did is to pass the first pair (s_t, a_t) of a sample to the network; then we integrated the output in the sample time dt, and added to the previous state (which is also the input of the network). In this way we obtain the state s_{t+1}. At this point we concatenate the new obtained state with the action a_{t+1} given by the sample dataset and we pass it to the network again, and repeating this process k times we get the states evolution according to the neural model.
Ended this mechanism, the loss is computed as the mean-squared error between the neural and the nominal evolutions.
The multi-step training is a crucial step for a non-physics informed model to learn the total evolution. Experiment in a single step prediction has shown that the displacement of the vehicle is to small for having a fine-grain error measurement.

5.1.2 Consideration of the Dataset

As anticiapted before, the dataset for such approach do not require any particular constraint. This allow us to test standard normalization techniques and so

robustify the model to different input scales. For doing so, we used a min-max normalization over the input, but we can also incorporate over the MLP architecture a normalization layer, that increase the stability of the train.
We prefered to use the layer norm in our experiment because such module do not require a preprocessing at inference time, before invoke the MPC framework. However, the normalization layer is still trained over the dataset, limiting the capacity of the network over range outside the training one.

5.1.3 Unicycle

In the application of the first approach over the unicycle model, the network was trained with 2 hidden state and a latent dimension of 128. The input was the concatenation of the state $x \in R^3$ and the action $u \in R^2$, while the output was the state derivative $\dot{x} \in R^3$.

5.1.4 Quadrotor

The quadrotor dynamic present a more complex model, with a state $x \in R^9$ and an action $u \in R^4$. For approximate such dynamic with the defined training scheme, a larger network as been tested: we increase the number of hidden layers up to 4. We have also tested the network with a latent dimension of 256, but the results were not competitive. In particular, considering the training time and the computational power required for the inference, we prefered to keep the same latent dimension of the unicycle network.

5.2 Unicycle Neural model

Aggiungere figura MLP

5.2.1 Architecture

The neural network used to model the kinematic model of the unicycle is a simple MLP architecture, with two hidden layers of 128 neurons. The network has two input which are concatenated together, the current state of the unicycle (dimension 3) and the current applied actions on the vehicle (dimension 2), so it must be of dimension 5 and is represented by

$$X = [x, y, \theta, v, \omega] \tag{23}$$

The output of the network has dimension 3, and it represents the dynamics of the system, therefore it reproduces the differential equations which describe the motion of the unicycle:

$$Y = [\dot{x}, \dot{y}, \dot{\theta}] \tag{24}$$

5.2.2 Dataset

The dataset was generated by using the nominal model of the robot and generating 500 trajectories. Each trajectory is composed by 1000 steps, where the inputs are maintained constant for the whole episode. This choice was taken because it was noticed that the interval between consecutive time-steps is too small, therefore the changes about the state of the unicycle are small too, and the network was not able to approximate correctly the motion of the vehicle. On the contrary, having the same input for a lot of steps allows to understand in a better way the dynamics of the system subject to specific inputs.
Each sample is not just a pair state-action at step t, but it is a window of dimension k which contains all pairs state-action from time-step t till $t+k$ (less the last action a_{t+k} which is useless), so it is in the form

$$(s_t, a_t, s_{t+1}, a_{t+1}, ..., s_{t+k-1}, a_{t+k-1}, s_{t+k}) \tag{25}$$

This allows to have a larger window of evolution of the system with respect to just the motion executed in one single step.

5.2.3 Training

As can be noticed in the previous section, the dataset does not contain the dynamics of the system, but just the evolution of the states. On the contrary, the forward pass of the network returns the vehicle dynamics. So, what we did is to pass the first pair (s_t, a_t) of a sample to the network; then we integrated the output in the sample time dt, and added to the previous state (which is also the input of the network). In this way we obtain the state s_{t+1}. At this point we concatenate the new obtained state with the action a_{t+1} given by the sample dataset and we pass it to the network again, and repeating this process k times we get the states evolution according to the neural model.
Ended this mechanism, the loss is computed as the mean-squared error between the neural and the nominal evolutions.

5.3 Quadrotor Neural model

6 Physical Informed Models

6.1 Physical Informed Neural Network: Introduction

Traditional methods for solving differential equations, such as finite element or finite difference methods, often require discretization of the domain and can be computationally expensive for high-dimensional problems. Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations. This approach should be able to generalize outside the training data, fully respect the underlying physics. With outside training data, we intend to say that the model generalize for sample that has never been shown **inside** the range of training.

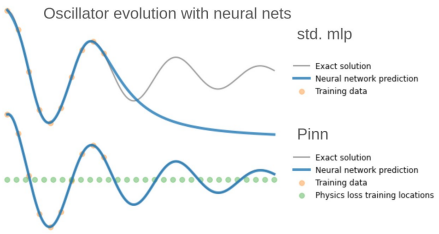

Figure 1: Example of the Pinn behavior.

The main loss component of the Pinn is call Physics Loss, and minimize the discrepancy between the dynamics of the system and the derivative of the network iteself.
The standard PiNN models takes in input just the time t at which we want to predict the evolution of the system. So, the output of the model is define as $x(t)$. For impose particular constrapngints, other losses are taken into account. Example of such loss are the so call *Boundary loss* and the *Initial loss*. The first one is used to impose the boundary condition of the system, while the second one is used to impose the initial condition of the system.

One limitation of such approach is that the model is not able to generalize to different initial condition: the model need to be trained again for predict a new evolution. Also, as few works have shown, the classica formulation of the Pinn is not stable over long time interval. Finally, the standard pinn model is not able to include the control input into the dynamics.
For this reason, we have used a extended version of the framework, namely, the PiNN-C (*PiNN with control* [1]). Such works shown as incorporate the control input on the prediction scheme, and has became the de-facto standard for model non-autonomus dynamic evolution ([2] [3]). The Pinn-C model is analyze in the next section of this report.

6.2 Physical Informed Neural Network with Control

The Pinn-c framework rely on the assumption that the input is constant over a sub-interval $[0, T_k]$. This assumption is used for ensure that the gradient of the output is not "corrupted" by the derivative of the state respect the input signal.

In this framework, the input of the network is composed by the initial condition of the sub-transition, the time stamp for which we want to obtain the prediction and the constant control input. Doing so, we obtain a set of tuple of the form $\{[x_0, \bar{u_k}, t_k], [x_{t_k}]\}$, where x_0 is the initial state of the system, t_k is the time stamp for which we want to predict the evolution and $\bar{u_k}$ is the control input applied to the system.
The pinn-c, in the ground, just model a set of smaller evolution, and the discriminative factor remains the time input, as in the standard formulation.

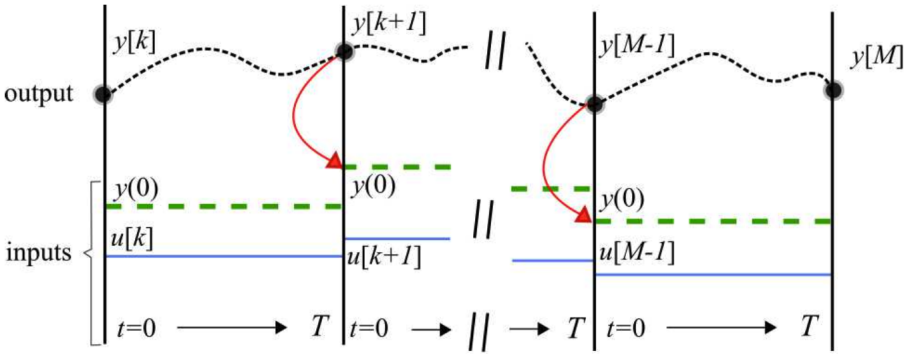

Figure 2: Pinn-c sub-interval integration

For concatenate the sub-trajectories, the starting continion of the next sub-trajectory is setted as the output of the previous interval at time T. For a concise representation, see figure 2 Finally, while the standard pinn can not rely on the trajectories from a dataset, the pinn-c must use it. This dependency is modeled as a imitation loss, similar to the standard MLP case.
In such framework it's not possible to rely on multi-step integration for calculate the loss term. Due the definition of input/output of the network, it's impossible to concatenate the previous integration into the future input. Extra details on the loss are given on the section below.

6.2.1 Dataset requirements, generation and constraints

In our experiment, after generate the evolution of the system, we notice that a good converage of the input space of the network is required. In the first phase of the experiment, we have generated a set of trajectories using a fixed family of input, like sinusoidal or step function. Although good result in the training and in the evaluation of the model (isolated form the mpc), we have notice that several phenomena conflict with the integration of the network into the mpc. In particular, the network was not able to predict the evolution of the system when the control

input was null. Also, basic motion like forward or backward trajectories remains challenging.
For this reason, we have decided to generate a new dataset, where the control input is randomly generated but all the possible combination of sing of the input are taken into account: we have generated 8 different combination for the Unicycle:

- $v > 0, w > 0$
- $v > 0, w < 0$
- $v < 0, w > 0$
- $v < 0, w < 0$
- $v > 0, w = 0$
- $v < 0, w = 0$
- $v = 0, w > 0$
- $v = 0, w < 0$

The last possible combination, $v = 0, w = 0$, has been excluded from the dataset, as we model a extra loss term that relies on sampling. The dataset has been generated using a Runge-Kutta 4th order method, and the sub-interval has been set to 0.1 seconds. In order to approximate the real evolution of the system, the PiNN-C framework needs different sample in which the input is constant. For this reason, after having discretize the input signal, we have taken up to 10 sub-evolution, leading to a reading of the trajectory of 0.02 seconds.
Under this assumption and constraint, we have seen that the previously mentioned phenomena are no longer present or at least has been minimized under a significant threshold.

6.2.2 Loss definition and training scheme

The overall loss function $\mathcal{L}$ used is composed by the sum of two main terms, which are the data loss (also call *imit loss*) $\mathcal{L}_{data}$ and the physics loss $\mathcal{L}_{phy}$.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{phy}}(\theta), \tag{26}$$

The first term defines the discrepancy between the prediction of the network and the observed data, while the second one specifies the correct modeling of the differential equation. Usually $\mathcal{L}_{data}$ is define as the mean squared error between the prediction and the ground truth, while $\mathcal{L}_{phy}$ is defined using the residual of the differential equation.
For a correct implementation of the physics loss, a sampling strategy is required. The usage of the physics loss can be seen as a self-supervised learning instance, due the fact that does not rely on any dataset.
At each iteration, the learner must sample a set of initial states, input and time from the relative distribution. Such samples are called "particles". A great number of sampled particle is important for a correct evaluation of the loss function, as

reported in the self-supervised learning literature. For such particle, we don't rely on the prediction of the network, but only on the gradient with respect to the input. Thanks to automatic differentiation, the gradient of the network can be easily calculated.
After having obtained the gradient, we can also calculate the output of the differential equation, and then calculate the loss term:

$$\mathcal{L}(\theta)_{physics} = \frac{1}{N}\sum_{i=1}^{N}(\nabla_x - ode(x,u,t))^2 \tag{27}$$

In the previous equation, *ode* is the differential equation that we are interested to learn as underlying prior, while $\nabla_(X)$ represents the gradient of the network with respect to the input state. The *ode* function is the same differential equation that we have introduced in the first part, and describes the evolution of our target systems.

An extra term has been added, as a new boundary condition. When the model has 0 input, the network must became an identity function, returning the same input state as output. This loss is defined as the mean squared error between the prediction and the input. Such application is possible only in the case of driftless system, as the unicycle. In the case of quadrotor, we prefer to include sample with 0 input in the dataset and rely on the integration of the dynamic iteself for incorporate such prior. This approach result in a less robust model, due the fact that the network only see few example of such evolution. In the case of driftless system, such loss can be implemented using a sampling strategy.
As in the physical loss, the input vector has been sampled in the correct range, then we manually zero the input element.
pipeline (prendi foto paper manipulator)

Algorithm 1 Pseudocode Representation

1: $\hat{y} \leftarrow \text{model}(X_{\text{data}})$
2: $L \leftarrow L + \text{mse}(y, \hat{y})$
3: Sample X
4: Calculate gradient J
5: Calculate analytical ode(X)
6: $L \leftarrow L + \backslash\text{equation}$
7: Sample point X
8: Zero the input

7 Results

what we have obtained

problems

pinn vs pinnc

8 Conclusions

References

[1] Eric Aislan Antonelo, Eduardo Camponogara, Laio Oriel Seman, Jean Panaioti Jordanou, Eduardo Rehbein de Souza, and Jomi Fred Hübner. Physics-informed neural nets for control of dynamical systems. *Neurocomputing*, 579:127419, April 2024.

[2] Jonas Nicodemus, Jonas Kneifl, Jörg Fehr, and Benjamin Unger. Physics-informed neural networks-based model predictive control for multi-link manipulators, 2021.

[3] Sourav Sanyal and Kaushik Roy. Ramp-net: A robust adaptive mpc for quadrotors via physics-informed neural network, 2023.